

Topic: Evolution of Distributed Systems

- From **1945 to mid-1980s**, computers were:
 - Large
 - Expensive
 - Not connected to each other
- Systems worked **independently** because:
 - Network technology was **very basic or not available**

Important Changes

These developments changed everything:

- **Powerful processors** → Faster and multi-core CPUs
- **High-speed networks** → LAN and WAN connections
- **Smaller devices** → Smartphones and IoT devices
- **Internet** → Global connection between computers

Definition of a Distributed System

Working Definition

A distributed system is a group of independent computers that **work together as one system**. Users see it as a single system, not many computers.

Two Key Characteristics

- **Autonomous computing elements**
 - Each computer (node) works **independently**.
 - It has its own **CPU, memory, and sometimes its own operating system**.
- **Single coherent system** coherent = uyumlu
 - Even if computers are separate, the system works like **one unified system**.
 - The user does not notice that there are many machines behind it.

Note

The definition is **flexible**.

It does not limit:

- hardware type
- network structure
- geographical distance

Characteristic 1: Autonomous Nodes

Key Properties

- Each node operates **independently**.
- Nodes communicate via **message passing** because they don't have shared memory.
- **No global clock** (each node has its own time)

Therefore, system must handle:

- Synchronization
- Coordination
- Membership management

✓ Result

- Now we can build systems with **many connected computers**
- These computers **work together like one system**

Real Life Examples

You use distributed systems when you use:

- Cloud storage
- Online banking
- Google Maps

Group Types

- **Open groups** → Any node may join the system.
- **Closed groups** → System membership is strictly controlled

Overlay Networks

In distributed systems, nodes are organized as **overlay networks**.

An overlay network is a **logical network built on a physical network**.

Types:

- **Structured overlays** → A network where each node has defined neighbors, such as a **ring** or tree topology. efficient search
- **Unstructured overlays** → A network where nodes **randomly** select their neighbors.
 - An important requirement is that the overlay must always remain connected.

Requirement

- Overlay must **stay connected**
- There must always be a **path between any two nodes**

Designing and maintaining this structure is **difficult**.

Characteristic 2: Single Coherent System

- The system should behave consistently regardless of location, time, or interaction mode.
- The system must **hide internal complexity from users**. Transparency !

However:

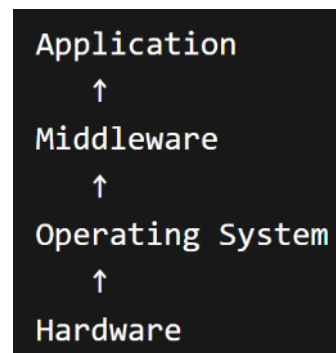
- **Partial failures are inescapable**
- **Hiding failures is very difficult**
- **Partial Failure:** A situation where some nodes fail while others continue operating, which is **inevitable** in distributed systems.

Middleware in Distributed Systems

- Distributed systems use **middleware to manage complexity**
- Middleware is a **layer** in the system

Middleware:

- Sits **above the operating system**
- It provides a **uniform interface** to applications
- It hides hardware and operating system heterogeneity (cesitlilik).



- It manages distributed resources, handles security and failures, and hides network complexity.

! Middleware vs. Operating System

Operating System: A software layer that manages the hardware resources of a **single** computer.

- Manages the CPU.
- Manages memory.
- Provides process communication.
- Controls hardware resources.

Middleware: A software layer that manages resources **across a distributed system**.

- Manages distributed resources.
- Provides inter-machine communication.
- Handles security and failures.
- Hides network complexity.

Middleware Analogy: Middleware is to distributed systems what an operating system is to a single computer.

Middleware in Distributed Systems

- Multiple computers (Computer 1–4)
- Each has its own **local operating system**
- A **distributed system layer (middleware)** spans all machines
- Applications run **above middleware**
- An application can be **distributed across multiple machines**
- Middleware **hides**:
 - Hardware differences
 - Operating system differences
 - Network communication details

Shortly It hides **Heterogeneity of system**.

! Examples of Middleware Services

Distributed systems often include a middleware layer.

1. **Remote Procedure Call (RPC):** Allowing you to call a function on a remote machine like it were local.
2. **Transactions:** Ensuring all-or-nothing execution across multiple machines. (banking systems)
3. **Service Composition:** Combining multiple services into a new application. (e-commerce)
4. **Reliability Services:** **Guaranteeing** message delivery.

! Major Design Goals

A distributed system should:

1. Support **resource sharing**
2. Provide **distribution transparency**
3. Be **open**
4. Be **scalable**

- These goals sometimes **conflict**
- A system optimized for transparency may have **performance loss**
- Designing distributed systems requires **balancing trade-offs**

Transparency is hard because of network delays, no global clock, and partial failures.

4. Major Design Goals

! Goal 1: Supporting Resource Sharing

- Systems share resources like files, storage, databases, services and devices.

Motivations:

- Economic efficiency → cheaper to share resources
- Collaboration → teams work together
- Data sharing → users access common data
- High availability → data is always accessible
- Modern cloud systems are examples
- Peer-to-peer systems (e.g., BitTorrent) show large-scale sharing

Goal 2: Distribution Transparency

- **Access Transparency:** Hiding data representation differences.
- **Location Transparency:** Hiding the physical location of a resource.
- **Migration Transparency:** Hiding the movement of resources.
- **Relocation Transparency:** Hiding the movement of resources during use.
- **Replication Transparency:** Hiding that multiple copies of a resource.
- **Concurrency Transparency:** Hiding simultaneous resource sharing.
- **Failure Transparency:** Hiding failures and system recovery.

Full transparency is difficult and may be expensive in performance.

- Goal: Hide that system components are **distributed**
- Example: When you open a URL, you don't know the **server location**

Trade-offs of Transparency

Full transparency is:

- Difficult
- Sometimes impossible
- Expensive in performance

There are physical limits:

- Communication speed is limited (speed of light)
- Wide-area networks have **latency**
- Strong consistency across systems takes **time**

Conclusion:

- Full transparency may **reduce performance**
- Transparency must be balanced with **performance and usability**
- Sometimes it is better to **show distribution** instead of hiding it
 - **Transparency is hard because of network delays, no global clock and partial failures.**

! Goal 3: Openness

Open Distributed System: A system that **uses standard interfaces and publicly defined rules.**

Interoperability: The ability of different components from different vendors to work together using agreed standards.

Portability: The ability to move software from one environment to another.

Extensibility: The ability to add new components

Mechanism: Defines **how** a capability is technically implemented. (How the system works.)

Policy: Defines **what** the system decides to do with a capability. (What the system decides.)

Separation of Policy and Mechanism

To achieve flexibility:

- Mechanism defines **how** the system is implemented
- Policy defines **what** decisions the system makes

Example:

- The system provides the ability to store and retrieve data (mechanism)
- The user or admin decides how this ability is used (policy)

Trade-off:

- Separating policy and mechanism increases flexibility but also increases complexity
- Finding the right balance is not easy

Goal 4: Scalability

Size Scalability: The ability to handle more users or resources.

Geographical Scalability: The ability to handle large physical **distances** between nodes.

Administrative Scalability: The ability for multiple organizations to manage the system.

- Modern systems must scale in all three dimensions.

A. Size scalability

- Centralized systems become bottlenecks due to:
 - CPU limitations
 - Storage I/O limits
 - Network bandwidth limits

Example:

- A video streaming system cannot support many users from a **single machine**
- As usage increases, **response time increases** significantly

B. Geographical Scalability

In wide-area systems:

- Latency increases significantly (WAN vs LAN)
- Communication is less reliable
- Bandwidth is more limited
- Communication methods that work in LAN may fail in WAN
- WAN systems require **asynchronous communication**

C. Administrative Scalability

- Large systems span **multiple organizations**

Each organization has:

- Different policies
- Different management domains
- Different security rules

Administrative scalability requires:

- Decentralized control
- Flexible trust mechanisms
- This is important in **cloud federations**

Fallacies of Distributed Systems



Assuming these statements are true will cause systems to fail in unexpected ways.

- The network is reliable.
- Latency is zero.
- Bandwidth is infinite.
- The network is secure.
- Topology does not change.
- There is only one administrator.

Why These Fallacies Are Dangerous

Consequences of wrong assumptions:

- Systems crash when **network problems** occur
- Applications fail under **high latency**
- **Security problems** can appear
- Scalability breaks under **heavy load**

Example:

- If we assume the network is reliable, we may not use retry mechanisms
- Then the system can crash when **packets are lost**

Conclusion:

- Distributed systems can fail in **unexpected ways** because partial failures are unavoidable

Partial Failure vs Centralized Failure

In centralized systems:

- The system either **works or completely crashes**
- Failure detection is **easy**

In distributed systems:

- Some nodes may **fail while others continue working**
- A node may be **slow or unreachable**
- It is hard to know if there is a **crashed node or a slow network** !!!

- This makes failure handling **more difficult**



6. Types of Distributed Systems



1. High-Performance Distributed Computing

- Focuses on maximizing computational power.
- The goal is parallel processing of large tasks.
- Examples : clusters, grid computing, and cloud computing.

! 2. Distributed Information Systems

- Focuses on data and transactions. Information dediği için data önemli = data consistent olmalı = para transferi
- Reliability and consistency are critical.
- Examples : enterprise systems, web applications, and banking systems.

! 3. Pervasive Systems (Ubiquitous Systems)

her yerde bulunan sistemler

- Systems embedded in everyday life with dynamic membership.

Characteristics:

- Mobile devices
- IoT environments
- Context awareness
- Dynamic membership

Mobile devices

- In pervasive distributed systems, many nodes are **mobile**
- It means ;
- They change their **physical location**
- They connect using **wireless networks**
- Their network connection can be **unstable**
- Their IP address may **change**

Examples:

- Smartphones
- Tablets

IoT Environments (Internet of Things)

- IoT environments include many **small embedded devices**
- They use **sensors and actuators**
- Devices have **limited CPU, memory, and battery**

Examples:

- Environmental sensors
- Industrial monitoring devices

Context Awareness

- The system adapts its behavior based on:
 - User location
 - Time of day
 - Device being used
 - Network conditions
 - Environmental conditions
- Context-aware systems do not treat all interactions the same

Example:

- Google Maps behaves differently if you are walking, driving, or on a train
- Smartphones adjust brightness based on light

Dynamic Membership

- Nodes can **join, leave, fail, or move** between networks
- There is no fixed set of participants

Example in a smart home:

- A guest connects to WiFi
- A new device is added
- A device stops working
- A sensor disconnects temporarily
- The system must **detect membership changes**

